

.Net Assemblies

- ➔ Microsoft .Net Assembly is a logical unit of code, it contains code that the Common Language Runtime (CLR) executes.
- ➔ Assembly is really a collection of types and resource information that are built to work together and form a logical unit of functionality.
- ➔ During the compile time Metadata is created, with Microsoft Intermediate Language (MSIL), and stored in a file called a Manifest
- ➔ Every Assembly you create contains one or more program files and a Manifest.
- ➔ There are two types program files :
 - Process Assemblies (EXE)
 - Library Assemblies (DLL).

Types of an Assembly

- (1) **Static assembly**
- (2) Dynamic assembly
- (3) **Private assembly**
- (4) Shared assembly

Static Assembly: The static assemblies are called exe [executable] and it store in the PE[Portable Executable] file.

Dynamic Assembly: These are called directly from the memory without utilize any resources of disc.

Private Assembly: A private assembly is use only by a single application and it store in install directory of the application. The private assembly has a unique name.

Shared Assembly: A shared assembly is work that can be reference by more than one application in folder to share the assembly must be explicitly build for this purpose by giving a cryptographically strong name.

Namespaces

- ➔ Namespaces are the way to organize .NET Framework Class Library into a logical grouping according to their functionality, usability as well as category they should belong to, or we cansay Namespaces are logical grouping of types for the purpose of identification.

There are two types of namespace:

(1) Inbuilt namespace

The namespaces which are already in the .NET Framework class library and which are not defined by user are called inbuilt namespaces.

(2) User defined namespace

The namespaces which are defined by any user as per requirement then those namespaces are known as user defined namespaces.

Types Of Common Namespaces ASP.Net**1) The System.Web namespace**

System.web namespace holds some basic ingredients which includes classes and built-in Objects like

- a. Server
- b. Application
- c. Request
- d. Response

2) The System.Web.services namespace

The System.Web.services namespace is a initial point for creating the web services. It contains web service classes , which allow to create XML web services using Asp.Net .ocol like SOAP , HTTP , XML .

Some classes are as follows

- a. WebMethodAttribute
- b. Webservice
- c. WebServiceAttribute
- d. WebServiceBindingAttribute

3) The System.web.UI.WebControls namespace

The System.web.UI.WebControls namespace contains classes that provides to create web server controls on web pages. Some classes are as follows

- a. Calendar
- b. Check Box
- c. Button
- d. Base Data Bound Control
- e. Data Control Field etc

4) The System.web.UI namespace

The System.web.UI namespace includes classes that allows to create server controls with data- binding functionality , which has ability to save view state of given control and pages (.aspx pages)

There are following controls available

- HTML server controls
- Web server controls
- User controls

5) The System.web.sessionstate namespace

Session state management comes under The System.web.sessionstate namespace . That enable storage of data

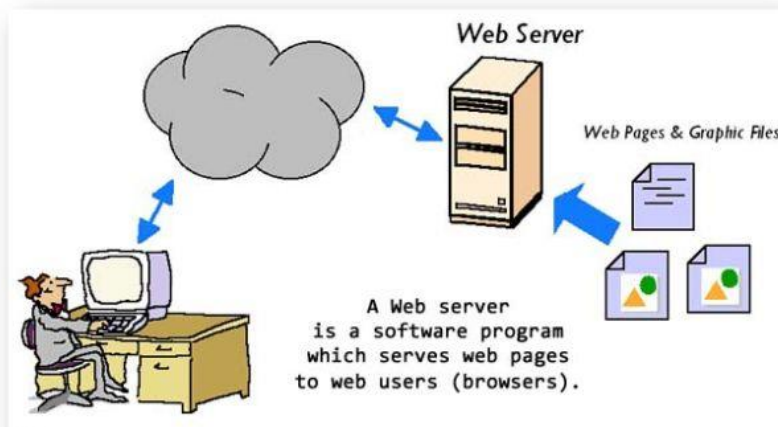
specific to a single client with in a web application on the server.

Some classes are as follows

- a. HttpSessionState
- b. HttpSessionStateContainer
- c. SessionIDManager
- d. SessionStateModule etc

Introduction to Web Server

- ➔ A web server is a software program that serves web pages to web users (browsers).
- ➔ A web server delivers requested web pages to users who enter the URL in a web browser.
- ➔ Every computer on the internet that contains a web site must have a web server program.



Characteristics of web servers

A web server computer is just like any other computer. The basic characteristics of web servers are:

- It is always connected to the internet so that clients can access the web pages hosted by the web server.
- It always has an application called "web server" running.

In short, a "web server" is a computer that is connected to the internet/intranet and has software called "web server". The web server program will always be running in the computer. When a user tries to access a website hosted by the web server, it is actually the web server program that delivers the web page that the client asks for.

All web sites in the internet are hosted in web servers sitting in various parts of the world.

-> Web Server hardware or software?

Mostly, Web server refers to the software program, that serves the clients request. But sometimes, the computer in which the web server program is installed is also called a "web server".



-> Web Server, Behind the Scenes

When I type in an URL such as <http://www.ASP.NET> and click on some link, I dropped into this page.

But what happens behind the scenes to bring you to this page and make you read this line of text.

The first you might do is, you type the <http://www.asp.net/> in the address bar of your browser and press your return key.

We could break this URL into the following two parts:

1. The protocol we will use to connect to the server (http)
2. The server name ([ASP.NET](http://www.asp.net/))

*****Web Based vs Window Based Application

Aspect	Web-Based Application	Window-Based Application
Platform	Web server	Local machine

******HTML
Controls
VS
ASP.NET**

Key Differences

Aspect	HTML Controls	ASP.NET Controls
Rendering	Client-side	Server-side
Event Handling	Client-side with JavaScript	Server-side with C# or VB.NET
State Management	Manual handling (cookies, sessions)	Automatic with ViewState
Data Binding	Not supported	Supported
Customization	Standard HTML attributes and CSS	Rich properties and events
Integration	Limited to client-side technologies	Full integration with ASP.NET framework
Performance	Fast rendering, client-side execution	Additional server round trips

achines

ites

urces

Controls,

1. HTML controls are standard HTML elements used to create the user interface in a web page. They are defined using HTML tags and are processed entirely on the client side by the web browser.

Characteristics

- **Standardization:** Follows the HTML standard and is supported by all web browsers.
- **Client-Side:** Rendered and processed on the client side.
- **Static Nature:** They do not have built-in server-side functionality.
- **Direct Manipulation:** Can be directly manipulated using JavaScript and CSS.

Examples

- **Button:**

```
html
<button type="button">Click Me!</button>
```

- **TextBox:**

```
html
<input type="text" id="txtName" name="txtName" />
```

- **DropDownList:**

```
html
<select id="ddlOptions">
    <option value="1">Option 1</option>
    <option value="2">Option 2</option>
</select>
```

ASP.NET Controls

ASP.NET controls are server-side controls provided by the ASP.NET framework.

They are more powerful than HTML controls as they integrate with server-side code, allowing for complex functionalities such as data binding, server-side events, and state management.

Characteristics

- **Server-Side:** Rendered and processed on the server side.
- **Rich Functionality:** Built-in functionalities like data binding, automatic state management (ViewState), and event handling.
- **ASP.NET Integration:** Integrates seamlessly with C# or VB.NET code for server-side processing.
- **Dynamic Nature:** Can be dynamically created and managed in the server-side code.

Examples

1. Button:

```
asp
<asp:Button ID="btnSubmit" runat="server" Text="Submit" OnClick="btnSubmit_Click" />
```

2. TextBox:

```
asp
<asp:TextBox ID="txtName" runat="server" />
```

3. DropDownList:

```
asp
<asp:DropDownList ID="ddlOptions" runat="server">
    <asp:ListItem Value="1">Option 1</asp:ListItem>
    <asp:ListItem Value="2">Option 2</asp:ListItem>
</asp:DropDownList>
```

****Understanding Standard Web Controls.

1. Label

This control is used to display textual information on the web forms. It is mainly used to create caption for the other controls like: textbox.

This is server side control, asp provides own tag to create label. The example is given below.

```
< asp:LabelID="Label1" runat="server" Text="Label" > </asp:Label>
```

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the label.
TabIndex	The tab order of the control.
BackColor	It is used to set background color of the label.
BorderColor	It is used to set border color of the label.
BorderWidth	It is used to set width of border of the label.

Font	It is used to set font for the label text.
ForeColor	It is used to set color of the label text.
Text	It is used to set text to be shown for the label.

Example

// WebControls.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebControls.aspx.cs"
Inherits="WebFormsControls.WebControls" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title></title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label ID="Label1" runat="server" Text="User Name"> </asp:Label>
      <asp:TextBox ID="TextBox1" runat="server" > </asp:TextBox> </td>

    </div>
  </form>
</body>
</html>
```

2. TextBox

This is an input control which is used to take user input. To create **TextBox** either we can write code or use the drag and drop facility of visual studio IDE.

This is server side control, asp provides own tag to create it. The example is given below.

1. <asp:TextBoxID="TextBox1" runat="server" ></asp:TextBox>

Server renders it as the HTML control and produces the following code to the browser.

1. <input name="TextBox1" id="TextBox1" type="text">

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.
BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
MaxLength	It is used to set maximum number of characters that can be entered.
ReadOnly	It is used to make control readonly.

Example

```

using System;
using System.Linq;
namespace WebFormsControlls
{
    public partial class WebControls : System.Web.UI.Page
    {
        protected void SubmitButton_Click(object sender, EventArgs e)
        {
            userInput.Text = UserName.Text;
        }
    }
}

```


}

3. Button

This control is used to perform events. It is also used to submit client request to the server. To create **Button** either we can write code or use the drag and drop facility of visual studio IDE.

This is a server side control and asp provides own tag to create it. The example is given below.

1. `<asp:ButtonID="Button1" runat="server" Text="Submit" BorderStyle="Solid" ToolTip="Submit"/>`

Server renders it as the HTML control and produces the following code to the browser.

1. `<input name="Button1" value="Submit" id="Button1" title="Submit" style="border-style:Solid;" type="submit">`

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.
BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.

Example

using System;

```

using System.Collections.Generic;
using System.Linq;
using System.Web;
namespace WebFormsControlls
{
    public partial class WebControls : System.Web.UI.Page
    {
        protected void Button1_Click(object sender, EventArgs e)
        {
            Label1.Text = "You Clicked the Button.";
        }
    }
}

```

4.HyperLink

It is a control that is used to create a hyperlink. It responds to a click event. We can use it to refer any web page on the server.

To create **HyperLink** either we can write code or use the drag and drop facility of visual studio IDE. This control is listed in the toolbox.

This is a server side control and ASP.NET provides own tag to create it. The example is given below.

1. `<asp:HyperLinkID="HyperLink1" runat="server" Text="JavaTpoint" NavigateUrl="www.javatpoint.com" ></asp:HyperLink>`

Server renders it as the HTML control and produces the following code to the browser.

1. `<a id="HyperLink1" href="www.javatpoint.com">JavaTpoint</a>`

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.

BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
NavigateUrl	It is used to set navigate URL.
Target	Target frame for the navigate url.

Example

```

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title> </title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:HyperLink ID="HyperLink1" runat="server" Text="Gmail" NavigateUrl="www.gmail.co
m"> </asp:HyperLink>
    </div>
  </form>
</body>
</html>

```

5. RadioButton

It is an input control which is used to takes input from the user. It allows user to select a choice from the group of choices.

To create **RadioButton** we can drag it from the toolbox of visual studio.

This is a server side control and ASP.NET provides own tag to create it. The example is given below.

1. `<asp:RadioButtonID="RadioButton1" runat="server" Text="Male" GroupName="gender"/>`

Server renders it as the HTML control and produces the following code to the browser.

1. `<input id="RadioButton1" type="radio" name="gender" value="RadioButton1" /><labelfor="RadioButton1">Male</label>`

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.
BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
GroupName	It is used to set name of the radio button group.

Example

```
using System;
using System.Collections.Generic;
namespace WebFormsControls
```

```

{
    public partial class WebControls : System.Web.UI.Page
    {
        protected void Button1_Click(object sender, EventArgs e)
        {
            genderId.Text = "";
            if (RadioButton1.Checked)
            {
                genderId.Text = "Your gender is " + RadioButton1.Text;
            }
            else genderId.Text = "Your gender is " + RadioButton2.Text;
        }
    }
}

```

6.

7. CheckBox

It is used to get multiple inputs from the user. It allows user to select choices from the set of choices.

It takes user input in yes or no format. It is useful when we want multiple choices from the user.

To create **CheckBox** we can drag it from the toolbox in visual studio.

This is a server side control and ASP.NET provides own tag to create it. The example is given below.

1. `< asp:CheckBox ID="CheckBox2" runat="server" Text="J2EE"/>`

Server renders it as the HTML control and produces the following code to the browser.

1. `< input id="CheckBox2" type="checkbox" name="CheckBox2" /> <label for="CheckBox2">J2EE</label>`

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.

BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
Checked	It is used to set check state of the control either true or false.

Example

```

<html >
<head >
    <title> </title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <h2>Select Courses</h2>
            <asp:CheckBox ID="CheckBox1" runat="server" Text="J2SE" />
            <asp:CheckBox ID="CheckBox2" runat="server" Text="J2EE" />
            <asp:CheckBox ID="CheckBox3" runat="server" Text="Spring" />
        </div>
        <p>
            <asp:Button ID="Button1" runat="server" Text="Button" OnClick="Button1_Click" />
        </p>
    </form>
    <p>
        Courses Selected: <asp:Label runat="server" ID="ShowCourses"> </asp:Label>
    </p>

```

`</body>``</html>`

8. LinkButton

It is a server web control that acts as a hyperlink. It is used to display a hyperlink-style button control on the web page. ASP.NET provides a tag to create LinkButton and has following syntax.

ASP.NET LinkButton Syntax

<asp:LinkButton

`AccessKey="string"``ID="string"``OnClick="Click event handler"``OnClientClick="string"``OnCommand="Command event handler"``OnDataBinding="DataBinding event handler"``OnDisposed="Disposed event handler"``OnInit="Init event handler"``OnLoad="Load event handler"``OnPreRender="PreRender event handler"``OnUnload="Unload event handler"``PostBackUrl="uri"``runat="server"``/>`

Example

Example

// LinkButtonExample.aspx

`<html >``<head >``<title> </title>``</head>``<body>``<form id="form1" runat="server">``<div>``<p>It is a hyperlink style button</p>``</div>`

```

<asp:LinkButton ID="LinkButton1" runat="server" OnClick="LinkButton1_Click">javatpoint</asp:LinkBut
ton>
<p>
    <asp:Label ID="Label1" runat="server"> </asp:Label>
</p>
</form>
</body>
</html>

```

Example

```

<form id="form1" runat="server">
<p>
    Click the button to download a file</p>
<asp:Button ID="Button1" runat="server" OnClick="Button1_Click" Text="Download" />
<br />
<br />
<asp:Label ID="Label1" runat="server"> </asp:Label>
</form>

```

*****Rich Control

1.FileUpload

It is an input controller which is used to upload file to the server. It creates a browse button on the form that pop up a window to select the file from the local machine.

To implement **FileUpload** we can drag it from the toolbox in visual studio.

This is a server side control and ASP.NET provides own tag to create it. The example is given below.

1. <asp:FileUpload ID="FileUpload1" runat="server"/>

Server renders it as the HTML control and produces the following code to the browser.

1. <input name="FileUpload1" id="FileUpload1" type="file">

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.
BackColor	It is used to set background color of the control.
BorderColor	It is used to set border color of the control.
BorderWidth	It is used to set width of border of the control.
Font	It is used to set font for the control text.
ForeColor	It is used to set color of the control text.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
AllowMultiple	It is used to allow upload multiple files by setting true or false.

Example

```

<html >
<head>
  <title> </title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <p>Browse to Upload File</p>
      <asp:FileUpload ID="FileUpload1" runat="server" />
    </div>
    <p>
      <asp:Button ID="Button1" runat="server" Text="Upload File" OnClick="Button1_Click" />
    </p>
  </form>
</body>
</html>

```

```

</form>
<p>
    <asp:Label runat="server" ID="FileUploadStatus"> </asp:Label>
</p>
</body>
</html>

```

2. Calendar

It is used to display selectable date in a calendar. It also shows data associated with specific date. This control displays a calendar through which users can move to any day in any year.

We can also set Selected Date property that shows specified date in the calendar.

To create **Calendar** we can drag it from the toolbox of visual studio.

This is a server side control and ASP.NET provides own tag to create it. The example is given below.

1. < asp:CalendarID="[Calendar1](#)" runat="[server](#)" SelectedDate="[2017-06-15](#)" > </asp:Calendar>

Server renders it as the HTML control and produces the following code to the browser.

This control has its own properties that are tabled below.

Property	Description
AccessKey	It is used to set keyboard shortcut for the control.
TabIndex	The tab order of the control.
Text	It is used to set text to be shown for the control.
ToolTip	It displays the text when mouse is over the control.
Visible	To set visibility of control on the form.
Height	It is used to set height of the control.
Width	It is used to set width of the control.
NextMonth Text	It is used to set text for the next month button.
TitleFormat	It sets format for month title in header.

DayHeaderStyle	It is used to set style for the day header row.
DayStyle	It is used to apply style to days.
NextPrevStyle	It is used to apply style to the month navigation buttons.

Example

```

using System;
using System.Collections.Generic;
namespace WebFormsControlls
{
    public partial class WebControls : System.Web.UI.Page
    {
        public void Calendar1_SelectionChanged(object sender, EventArgs e)
        {
            ShowDate.Text = "You Selected: "+Calendar1.SelectedDate.ToString("D");
        }
    }
}

```

3. GridView Control

Definition

The GridView control displays the values of a data source in a table where each column represents a field and each row represents a record.

Syntax

```
<asp:GridView ID="GridView1" runat="server"> </asp:GridView>
```

Example

```
<asp:GridView ID="GridView1" runat="server" AutoGenerateColumns="True" />
```

Code-Behind

csharp

```
using System;
```

```
using System.Web.UI;
```

```
namespace RichControlsExample
```

```
{
```

```
    public partial class _Default : Page
```

```
    {
```

```
        protected void Page_Load(object sender, EventArgs e)
```

```

    {
        if (!IsPostBack)
        {
            GridView1.DataSource = GetData();
            GridView1.DataBind();
        }
    }

    private object GetData()
    {
        return new[]
        {
            new { ID = 1, Name = "John Doe", Age = 30 },
            new { ID = 2, Name = "Jane Smith", Age = 25 }
        };
    }
}

```

4. DataList Control

Definition

The DataList control displays data in a format that you can define using templates and styles.

Syntax

```
<asp:DataList ID="DataList1" runat="server"> </asp:DataList>
```

Example

aspx

Copy code

```

<asp:DataList ID="DataList1" runat="server">
    <ItemTemplate>
        <div>
            ID: <%# Eval("ID") %> <br />
            Name: <%# Eval("Name") %> <br />
            Age: <%# Eval("Age") %>
        </div>
    </ItemTemplate>
</asp:DataList>

```

Code-Behind

```
csharp
Copy code
using System;
using System.Web.UI;

namespace RichControlsExample
{
    public partial class _Default : Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                DataList1.DataSource = GetData();
                DataList1.DataBind();
            }
        }

        private object GetData()
        {
            return new[]
            {
                new { ID = 1, Name = "John Doe", Age = 30 },
                new { ID = 2, Name = "Jane Smith", Age = 25 }
            };
        }
    }
}
```